UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Software Architecture.

The shipped system: surfaces, components, runtime data flows, and the audit substrate behind every GLM decision.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

SAD · SOFTWARE ARCHITECTURE APRIL 2026 SUBMISSION RELEASE

1. System Overview

Opus Magnum is a three-surface, single-engine platform:

- **Admin surface** (`/admin/*`) — templates authored in a UI, with GLM in the loop.
- **Owner surface** (`/onboard/*` , `/dashboard`) — runs a city's template step by step; watches GLM process every document.
- **Visitor surface** (`/buildings` , `/buildings/[id]`) — browses published buildings in 3D, asks a grounded assistant.

Under the surfaces is one **adaptive onboarding engine**: a LangGraph StateGraph compiled from a JSON template, with GLM invoked at every reasoning node and the full trail persisted for audit.

2. Component Diagram

```
┌──────────────────────────────────────────────────────────────────────────┐
│                       Frontend (Next.js 16, TS)                          │
│                                                                          │
│   Admin UI            Owner UI                Visitor UI                 │
│   /admin/*            /onboard/[id]           /buildings/[id]            │
│   • template editor   • workflow canvas       • 3D scene (R3F + GLB)     │
│   • template asst     • step-form / upload    • grounded assistant       │
│                       • decision-log drawer                              │
└──────────┬──────────────────────┬──────────────────────┬──────────────────┘
           │            HTTP + SSE (CORS)                │
           ▼                      ▼                      ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                  Backend  (FastAPI, Python 3.12)                         │
│                                                                          │
│   /api/admin/*      /api/onboard/*         /api/buildings/*              │
│                                                                          │
│    ┌──────────────────────────────────────────────────────────────────┐  │
│    │              GLM Reasoning Core (ILMU API)                       │  │
│    │   OpenAI SDK → https://api.ilmu.ai/v1  (ilmu-glm-5.1)            │  │
│    │                                                                  │  │
│    │   Tools exposed to GLM:                                          │  │
│    │    draft_template, explain_field, extract_document,              │  │
│    │    verify_against_criteria, cross_check_documents,               │  │
│    │    summarize_for_review, answer_visitor_question                 │  │
│    │                                                                  │  │
│    │   Every call → Decision → step_runs.decision_log                 │  │
│    └──────────────────────────────────────────────────────────────────┘  │
│                                                                          │
│    ┌────────────────────────┐      ┌──────────────────────────────┐      │
│    │   LangGraph Runtime    │◄────►│    Template Compiler         │      │
│    │   StateGraph per run   │      │    JSON steps → nodes+edges  │      │
│    │   SQLite/Postgres      │      │                              │      │
│    │   checkpointer         │      │    Primitives registry       │      │
│    └────────────────────────┘      └──────────────────────────────┘      │
│                                                                          │
│    ┌────────────────────────┐      ┌──────────────────────────────┐      │
│    │  Doc Processing Hub    │      │  Building Profile Builder    │      │
│    │  pdf → text (pypdf +   │      │  content-doc extractions     │      │
│    │  OCR fallback)         │      │   → profile + scene_config   │      │
│    │  image → OCR           │      │                              │      │
│    └────────────────────────┘      └──────────────────────────────┘      │
└──────────────────────────────────────────────────────────────────────────┘
           │
           ▼
   ┌──────────────────────────────────────────────────────────────────┐
   │  Postgres / SQLite                                               │
   │  users, cities, workflow_templates, buildings, workflow_runs,    │
```

```
    │  step_runs, documents                                          │
    │                                                                 │
    │  Object store: local filesystem (dev) or S3 (prod path)         │
    └─────────────────────────────────────────────────────────────────┘
```

3. Runtime Data Flow

3.1 Admin — authoring a template

1. Admin enters a one-line description in the template editor.
2. `POST /api/admin/templates/{id}/draft-with-ai` invokes `GLMClient.call_tool` with the `draft_template` function schema, using `DRAFT_TEMPLATE_SYSTEM` as the system prompt.
3. GLM returns a structured `steps: [...]` array conforming to the six allowed primitives.
4. Admin tweaks fields in the editor (drag-reorder, primitive-specific config, criteria list).
5. `POST /api/admin/templates/{id}/publish` flips `status='published'`.
6. `POST /api/admin/templates/{id}/assistant` streams tutor-style answers via SSE when the admin asks a question about any step.

3.2 Owner — running a workflow

1. Owner creates a building and calls `POST /api/onboard/buildings/{id}/start`, which creates a `workflow_runs` row and invokes the runner.
2. The runner:
3. loads `WorkflowTemplate.steps`,
4. compiles them into a `StateGraph` via `compile_template(steps)`,
5. opens the SQLite/Postgres checkpointer (thread_id = run_id),
6. invokes the graph with the current `RunState`.
7. Each primitive node inspects `state.pending_input`; if nothing is queued, it returns `awaiting_user=True` and the conditional edge `_route_after_step` routes to `END`, pausing the graph.
8. The API surfaces the paused state: `GET /run` returns `RunState`, `GET /run/graph` returns a React-Flow-friendly shape.
9. When the owner submits a form or uploads documents, the corresponding endpoint merges the payload into `pending_input[step_id]` and calls `tick()`, which re-invokes the graph — the primitive now has input and proceeds.
10. Upload endpoints stream multipart files through `processing/pipeline.py` (text extraction via `pypdf` or OCR), persisting `Document` rows.

11. Every GLM call inside a primitive passes through a `Decision` sink that tags each entry with `step_id` and appends it to `state.decision_log`. The runner mirrors these into the `step_runs.decision_log` column when it persists the snapshot.
12. When the `publish` primitive fires, the runner calls `build_profile(state)` and writes `profile` + `scene_config` onto the `Building` row.

3.3 Visitor — asking a grounded question

1. Visitor opens a published building; the frontend fetches `/api/buildings/{id}`.
2. Chat sends `POST /api/buildings/{id}/assistant` with the message history.
3. The backend assembles grounding material: `building.profile`, `building.scene_config`, and content-doc extracts (budgeted to ≤ 60k chars).
4. `GLMClient.complete(stream=True)` with `ANSWER_VISITOR_QUESTION_SYSTEM` streams tokens back via `text/event-stream`.

4. Data Model (summary)

TABLE	PURPOSE	KEY COLUMNS
`users`	seeded admin + owner (mock auth)	`id`, `email`, `role`
`cities`	jurisdiction keyed by name	`id`, `name`, `country`, `region`
`workflow_templates`	admin-authored JSON template	`id`, `city_id`, `steps (JSON)`, `status`
`buildings`	onboarding + publication record	`id`, `owner_id`, `city_id`, `status`, `profile (JSON)`, `scene_config (JSON)`
`workflow_runs`	one per building onboarding	`id`, `building_id`, `template_id`, `state (JSON)`, `status`, `current_step_id`
`step_runs`	per-step audit trail	`run_id`, `step_id`, `primitive`, `output (JSON)`, `decision_log (JSON)`, `status`
`documents`	uploaded files + extractions	`id`, `building_id`, `step_run_id`, `doc_class`, `file_url`, `extracted (JSON)`, `verification_result (JSON)`

Two JSON stores that evolve independently. `workflow_templates.steps` holds the domain-specific authoring shape; `workflow_runs.state` holds the frozen `RunState` snapshot. Both are JSONB on Postgres and TEXT on SQLite — same code either way.

`decision_log` is the AI audit substrate. Every row in `step_runs.decision_log` is one GLM interaction: call_id, tool name, request messages (with long content elided), response content + tool_calls + usage, duration. Snapshot tests compare the shape as a regression net against silent GLM behaviour drift.

5. Module Boundaries

MODULE	RESPONSIBILITY
`backend/app/glm/`	Single point of contact with the ILMU API. Tool schemas, prompts, decision-log emission. Nothing outside this module talks to OpenAI.
`backend/app/workflow/`	Template compilation, RunState, primitives, the runner.
`backend/app/workflow/primitives/`	One file per primitive. Each exports a factory that returns the graph node. New primitives = new file + registry decorator.
`backend/app/processing/`	PDF + image parsing. Returns plain text. Never calls GLM.
`backend/app/profile/`	Pure function: `RunState` → (`profile` , `scene_config`). No side effects.
`backend/app/db/`	SQLAlchemy models, session, `create_all` schema bootstrap.
`backend/app/api/`	FastAPI routers. Thin — delegate all logic to the modules above.
`frontend/lib/`	Typed API client (`api.ts`), SSE helper (`sse.ts`).
`frontend/components/onboard/`	Workflow canvas, step forms, upload panel, decision log.
`frontend/components/building/`	3D scene, assistant chat.
`frontend/components/admin/`	Template editor, template assistant.

A reviewer should be able to answer, for any module, three questions: *what does it do*, *how is it used*, *what does it depend on*, from the first thirty lines of the primary file.

6. Interfaces

6.1 HTTP (JSON except where noted)

```
POST /api/admin/cities                                    → create city
GET  /api/admin/cities                                    → list cities
POST /api/admin/cities/{cityId}/templates                 → create template
GET  /api/admin/templates/{id}                            → read template
PUT  /api/admin/templates/{id}                            → update steps
POST /api/admin/templates/{id}/publish                    → flip to published
POST /api/admin/templates/{id}/draft-with-ai              → GLM drafts steps
POST /api/admin/templates/{id}/assistant                  → SSE, tutor chat

POST /api/onboard/buildings                               → create building
GET  /api/onboard/buildings                               → list owner's buildings
POST /api/onboard/buildings/{id}/start                    → create run + tick
GET  /api/onboard/buildings/{id}/run                      → RunState snapshot
GET  /api/onboard/buildings/{id}/run/graph                → React-Flow nodes + edg
GET  /api/onboard/buildings/{id}/run/stream               → SSE progress events
POST /api/onboard/buildings/{id}/run/steps/{stepId}/submit→ form / ack payload
POST /api/onboard/buildings/{id}/run/steps/{stepId}/upload→ multipart files
POST /api/onboard/buildings/{id}/publish                  → force publish

GET  /api/buildings                                       → published list
GET  /api/buildings/{id}                                  → profile + scene_config
POST /api/buildings/{id}/assistant                        → SSE grounded answer
```

6.2 GLM tool schemas (OpenAI function-calling)

Seven tools, one system prompt each. See `implementation.md §9`, mirrored in `backend/app/glm/tools.py`:

`draft_template`, `explain_field`, `extract_document`, `verify_against_criteria`, `cross_check_documents`, `summarize_for_review`, `answer_visitor_question`.

7. Process Model

The backend is a single FastAPI process (uvicorn). Each HTTP request handler is stateless; all persistence is in Postgres/SQLite (app data) or the LangGraph checkpointer (graph state). A graph invocation compiles fresh each tick — compile is cheap and avoids stale checkpointer references across requests.

Long-lived streams are SSE (GET for visitor/run-progress; POST-via-fetch for assistant chat and template assistant). There are no WebSockets.

8. Failure Modes & Recovery

FAILURE	DETECTED IN	RECOVERY
GLM 429 rate limit	`GLMClient._chat`	Exponential backoff with jitter, up to 4 retries
GLM 5xx	`GLMClient._chat`	Same retry loop
GLM returns invalid JSON	`complete_json` / `call_tool`	Returns `{_error, _raw}`; caller treats as failure without crashing
Primitive raises	`runner.tick`	Catches, persists `status='failed'`, returns snapshot; UI shows error badge
Verification fails (criterion unmet)	`upload_compliance` primitive	Graph pauses on same step; owner re-uploads
OCR binary missing	`processing/pdf.py`, `processing/image.py`	Returns empty string; `decision_log` marks `ocr_used=false` so reviewer can act
Scanned PDF with no text layer	`extract_pdf_text`	Automatic `pdf2image + pytesseract` fallback if extras installed
Owner abandons run	n/a	`RunState` is checkpointed; next `tick()` resumes from paused step

9. Non-functional Properties

Auditability. Every GLM decision is persisted with call_id, duration, input, and output. This is the single highest-value non-functional property — the platform's value proposition rests on judged decisions being inspectable.

Resumability. Runs survive process restarts via the LangGraph checkpointer. Owner can return days later and continue.

Determinism of the template model. Template JSON → StateGraph → same edges every time. The compiler is a pure function of its input.

Deployability. Backend: one Python service + one database. Frontend: static Next.js. No bespoke infrastructure; free tiers of Neon + Vercel + Koyeb cover the MVP.

See `tad.md` for the technology-selection rationale and `implementation.md` for the authoritative engineering source of truth.